

UNIVERSITÀ DEGLI STUDI DI CATANIA
Anno Accademico 2024 - 2025
Corso di Laurea in Informatica
Prova scritta di Programmazione 1 e laboratorio (A-E / O-Z)
15/04/25

COGNOME e NOME: (IN STAMPATELLO)	
N. MATRICOLA:	

**Non sono consentiti formulari, appunti, libri e calcolatori; non è consentito comunicare con i colleghi; ogni mezzo di comunicazione elettronico deve essere tenuto spento.
Il test va sempre consegnato prima dell'uscita dall'aula.**

Per ciascuna delle seguenti venti domande indicare l'unica risposta corretta.
La prova si considera superata con almeno undici risposte corrette.

1) Sia definita la seguente funzione

```
void func(char *str, const char *source){
    int i=0;
    while(*source)
        str[i++]=*(source++);
    str[i] = 0;
}
```

- ☐ la funzione copia la stringa `str` nell'area di memoria puntata da `source`;
- ☐ la funzione copia `i` caratteri della stringa `source` in `str`;
- ☒ la funzione copia la stringa `source` nell'area di memoria puntata da `str`;
- ☐ la funzione contiene un errore;
- ☐ nessuna delle risposte precedenti è corretta;

2) Completare il seguente frame di codice con l'unica istruzione corretta:

```
struct record{
    char **vec[25];
}I1;
// ..
???
```

- ☐ `printf("%s", I1.*vec);`
- ☐ `printf("%s", *(I1.vec[rand()%25]));`
- ☐ `printf("%s", I1.*vec[rand()%25]);`
- ☐ `printf("%s", I1.vec[rand()%25]);`
- ☒ `printf("%s", *I1->vec[rand()%25]);`

3) Per la seguente istruzione:

```
printf("%zd", sizeof(int));
```

- ☒ il corrispondente output dipenderà dalla piattaforma;
 - ☐ il corrispondente output sarà 4;
 - ☐ il corrispondente output sarà 8;
 - ☐ il corrispondente output sarà 12;
 - ☐ nessuna delle risposte precedenti è corretta;
-

4) Quante iterazioni saranno prodotte dal seguente ciclo for:

```
for(int x = -4; x<=9; x++)  
    //..
```

- ☐ un numero di iterazioni indefinito;
 - ☒ produrrà 14 iterazioni;
 - ☐ produrrà 13 iterazioni;
 - ☐ produrrà 15 iterazioni;
 - ☐ nessuna delle risposte precedenti è corretta;
-

5) La funzione di libreria `fopen()`

- ☐ restituisce un puntatore ad `int`, oppure `NULL` in caso di errore;
 - ☐ provoca errore di segmentazione se il file indicato tra gli argomenti non esiste;
 - ☒ restituisce un puntatore ad una struttura di un tipo ben definito, oppure `NULL` in caso di errore;
 - ☐ restituisce un puntatore a `void` oppure `NULL` in caso di errore;
-

6) Il seguente frame di codice

```
double A[10][20];  
double B[10][20];  
A=B;  
A[5][5] = 1.2;
```

- ☐ provoca un memory leak;
 - ☐ provoca effetto aliasing;
 - ☒ provoca errore di compilazione;
 - ☐ provoca errore a tempo di esecuzione;
-

7) Con riferimento al seguente frame di codice, che contiene una invocazione della funzione `char *strcpy(char *dst, const char * src)`

```
char *str = "abcdefg";  
char *str1 = malloc(sizeof(char)*20);  
if(str1)  
    strcpy(str1, str);
```

- ☐ dopo l'esecuzione, la stringa `str` sarà copiata in `str1`;
 - ☐ si avrà un errore di segmentazione, in quanto l'area di memoria puntata da `str1` è troppo ampia rispetto alla lunghezza della stringa `str`;
 - ☒ il frame di codice è incompleto, in quanto la stringa `str1` deve terminare con il carattere di terminazione stringa, che non viene inserito dalla funzione `strcpy`;
 - ☐ errore a tempo di esecuzione;
 - ☐ nessuna delle precedenti risposte è corretta;
-

8) È possibile allocare un vettore di n elementi di tipo `double**` nel seguente modo:

- ☐ `double **M = malloc(sizeof(double**)*n)`
 - ☒ `double ***M = malloc(sizeof(double**)*n)`
 - ☐ `double **M[n] = malloc(sizeof(double**)*n)`
 - ☐ `double *M[n] = malloc(sizeof(double**)*n)`
-

9) con riferimento alla struttura dati CODA implementata come lista dinamica:

- ☐ è necessario mantenere un unico puntatore all'elemento in coda;
 - ☐ l'inserimento di un elemento richiede una visita completa della coda a partire dalla testa;
 - ☐ la cancellazione di un elemento richiede il puntatore alla coda;
 - ☐ l'inserimento di un nuovo elemento richiede il puntatore alla testa;
 - ☒ nessuna delle precedenti risposte è corretta;
-

10) Scegliere un'opzione per la compilazione del seguente frame di codice. Si supponga che tale frame di codice costituisca una translation unit di nome `test.c` e che all'interno del file non vi è altro codice.

```
void func(int a, double a2);
void bla(){
    func(1, 4.5);
}
```

- ☐ `$ gcc -Wall test.c`
 - ☐ `$ gcc -Wall -E test.c`
 - ☒ `$ gcc -Wall -c test.c`
 - ☐ nessuna delle risposte precedenti è corretta.
-

11) Completare il seguente frame di codice:

```
struct record{
    double x;
    //...
}rec;
???
```

- ☐ `scanf("%lf", &(rec.x));`
 - ☐ `scanf("%f", &(rec->x));`
 - ☐ `scanf("%lf", &(rec->x));`
 - ☒ `scanf("%lf", rec.x);`
-

12) Completare il seguente frame di codice con una delle quattro proposte elencate in seguito

```
void bar(unsigned K, unsigned L, int ***V){
    for(unsigned i=0; i<K; i++)
        for(unsigned j=0; j<L; j++)
            ??? = rand()%10000;
}
```

- ☒ `**(V[i]+j)`
 - ☐ `**V[i+j]`
 - ☐ `**V[i]+j`
 - ☐ `V[i][j]`
-

13) Sia `source.c` un file sorgente C. Indicare il comando apposito che permetta di ottenere, su standard output, il risultato della fase di pre-processing (output del pre-processore):

- ☐ `$ gcc source.c -o source.S`
 - ☒ `$ gcc -E source.c`
 - ☐ `$ gcc -c source.c`
 - ☐ `$ gcc -c source.c -E source.S`
-

14) Completare opportunamente il seguente frame di codice selezionando una delle opzioni indicate nel seguito:

```
void foo(char **W, int ns){
    for(int i=0; i<ns; i++){
        char *pointer = W[i];
        ???
    }
}
```

- ☐ `while(++pointer) {printf("%c", *pointer);}`
 - ☐ `while(pointer++) {printf("%c", *pointer);}`
 - ☐ `while(*pointer) {printf("%c", *(pointer+1));}`
 - ☒ `while(*pointer) {printf("%c", *(pointer++));}`
-

15) In corrispondenza del seguente frame di codice:

```
#define N 10
//...
double *x = malloc(sizeof(double)*N);
//..
free(x)
if(x!=NULL)
    free(x)
```

- ☐ si avrà errore di compilazione;
 - ☒ errore a tempo di esecuzione;
 - ☐ nessun errore;
 - ☐ warning del compilatore;
-

16) Sia W il nome di un array monodimensionale precedentemente allocato nel segmento STACK (allocazione automatica):

- ☐ W sarà deallocato automaticamente solo quando il programma avrà terminato la sua esecuzione;
 - ☐ W potrà essere deallocato invocando la funzione `free()`;
 - ☒ W sarà deallocato a seguito dell'uscita, del flusso di esecuzione, dalla funzione in cui W è stato dichiarato/allocato;
 - ☐ nessuna delle risposte precedenti è corretta.
-

17) Nella codifica di un numero in virgola mobile (standard IEEE 754 a 64 bit, ovvero doppia precisione):

- ☐ non vi è alcun bit che rappresenta il segno, il quale è rappresentato implicitamente nel campo esponente;
 - ☐ la cosiddetta precisione P è uguale a 6;
 - ☒ la cosiddetta precisione P è uguale a 15;
 - ☐ l'intervallo rappresentabile è uguale a $[-2^{64} - 1, 2^{64} - 1]$;
 - ☐ nessuna delle risposte precedenti è corretta;
-

18) In corrispondenza della seguente istruzione:

```
double y = FLT_MAX * FLT_MAX;
```

- ☐ si avrebbe errore in fase di esecuzione;
 - ☐ si avrebbe errore in fase di compilazione;
 - ☒ si avrebbe warning in fase di compilazione;
 - ☐ nessuna delle precedenti risposta è corretta;
-

19) Il selection sort:

- ☐ opera senza alcuno scambio tra alcuno degli elementi dell'array;
 - ☐ opera mediante opportuni scambi tra elementi adiacenti nell'array;
 - ☐ opera mediante un array di appoggio;
 - ☒ opera mediante selezione dell'elemento minimo/massimo del sotto-array non ordinato;
-

20) Il seguente frame di codice

```
// sia V una costante positiva
unsigned i=0;
while( i++<V )
    printf("\n %u", rand()-5000);
```

- ☐ permetterebbe di stampare $V + 1$ numeri pseudo-casuali;
- ☐ permetterebbe di stampare $V - 1$ numeri pseudo-casuali;
- ☐ permetterebbe di stampare V numeri pseudo-casuali;
- ☒ contiene un errore;